



BRIDGELABZ JAVA PROGRAMMING FUNDAMENTALS IMMERSION MAKER PLAN
— 160 HRS

Stages - Core Java Programming, Object Oriented Programming with Java (OOP), Data Structure & Algorithms (DSA)

Core Programming (9 Days) | OOP with Java (11 Days) | DSA (15 Days) | 32 Sessions + 3 Workshops | Platform: CodIn

Fellowship Part	Day #	Session Type	Concept Introduced	Story-Based Sample Problem Statements (Daily Practice)	LeetCode Sample Problem References
Core Programming	1	Regular	Java Environment Setup, JVM/JDK/JRE architecture, Data Types, Variables, Type Casting, Operators Proper Programming Hygiene GIT Introduction: Git vs GitHub, Commits, Version History and Messages	Ravi just joined a coding guild. Store his name, age, rank, salary, and membership fee (float) using correct data types. Compute his annual bonus (12% of salary), cast it to int, and print a formatted welcome card.	LC #1 Palindrome Number LC #2 Roman to Integer LC #3 Subtract the Product and Sum of Digits
Core Programming	2	Regular	Control Flow: if-else, ternary, switch; Loops: for, while, do-while, break, continue	A kingdom's tax system charges 5% for income <10K, 15% for 10K–50K, 30% above 50K. Read a citizen's income, print tax bracket and amount. Extend: loop over an array of 10 citizens and tally total tax collected.	LC #4 FizzBuzz LC #5 Reverse Integer LC #6 Plus One LC #7 Number of Steps to Reduce to Zero
Core Programming	3	Regular	Arrays: 1D & 2D, traversal, min/max, linear search, frequency count, rotation	A warehouse stores item quantities in an array. Find the max, min, total stock, and detect duplicates. Extend: rotate the stock array by k positions (simulate daily shift handover), and transpose a 2D shelf grid.	LC #8 Contains Duplicate LC #9 Majority Element LC #10 Max Consecutive Ones LC #11 Move Zeroes LC #12 Rotate Array
Core Programming	4	Regular	Strings: String methods, StringBuilder, immutability, palindrome, anagram, character frequency	A spy agency encodes messages. Reverse a secret message, check if it is a palindrome, count vowels/consonants, and verify if two intercepts are anagrams. Extend: find the first non-repeating character in a surveillance log.	LC #13 Valid Palindrome LC #14 Valid Anagram LC #15 First Unique Character in String LC #16 Reverse String LC #17 Longest Common Prefix



BRIDGELABZ JAVA PROGRAMMING FUNDAMENTALS IMMERSION MAKER PLAN
— 160 HRS

Stages - Core Java Programming, Object Oriented Programming with Java (OOP), Data Structure & Algorithms (DSA)

Core Programming (9 Days) | OOP with Java (11 Days) | DSA (15 Days) | 32 Sessions + 3 Workshops | Platform: CodIn

Fellowship Part	Day #	Session Type	Concept Introduced	Story-Based Sample Problem Statements (Daily Practice)	LeetCode Sample Problem References
Core Programming	5	Regular	Functions & Methods: parameters, return types, method overloading, pass-by-value, variable scope	Build a 'Math Wizard' utility class: isPrime(n), factorial(n) using iteration, fibonacci(n), gcd(a,b), lcm(a,b), and power(base, exp). Overload factorial to also accept double. Demonstrate scope differences between local and instance variables.	LC #18 Power of Two LC #19 The kth Factor of n LC #20 Fibonacci Number LC #21 GCD of Strings
Core Programming	6	Regular	Recursion: base case, recursive case, recursion tree, stack overflow awareness	A temple has N disks in the Tower of Hanoi puzzle. Solve it recursively and count moves. Also implement recursive: binary search on a sorted price list, sum of digits, reverse a string, and check balanced parentheses without a stack.	LC #22 Fibonacci Number LC #23 Climbing Stairs LC #24 Reverse Linked List (recursive preview) LC #25 Reverse String
Core Programming	7	Regular	Exception Handling: try-catch-finally, checked vs unchecked, custom exceptions, throws	A hospital billing system must never crash. Wrap division-by-zero (bills with zero items), array out-of-bounds (invalid patient index), and number format exceptions (bad input) with meaningful messages. Create a custom InsufficientFundsException for payment failures.	LC #26 Palindrome Number (input validation) HackerRank Java Exception Handling series
Core Programming	8	Regular	Java I/O: Scanner, BufferedReader, File I/O (FileReader/FileWriter), basic serialization	A school result portal reads student names and marks from a text file, computes averages, and writes a formatted report card to a new file. Handle FileNotFoundException gracefully. Extend: append new results without overwriting old ones.	LC #27 Find Index of First Occurrence (string parsing) File I/O design exercise



BRIDGELABZ JAVA PROGRAMMING FUNDAMENTALS IMMERSION MAKER PLAN
— 160 HRS

Stages - Core Java Programming, Object Oriented Programming with Java (OOP), Data Structure & Algorithms (DSA)

Core Programming (9 Days) | OOP with Java (11 Days) | DSA (15 Days) | 32 Sessions + 3 Workshops | Platform: CodIn

Fellowship Part	Day #	Session Type	Concept Introduced	Story-Based Sample Problem Statements (Daily Practice)	LeetCode Sample Problem References
Core Programming	9	Workshop	WORKSHOP 1: Core Programming — Integration Challenge	Build a command-line 'Student Grade Manager'. Requirements: read student data from a file (name, marks in 5 subjects), compute grade using nested conditions, store in arrays, search by name, sort by total marks (bubble sort), write report to output file. Must use methods, exceptions, and file I/O.	LC #28 Merge Sorted Array LC #29 Pascal's Triangle LC #30 Plus One LC #31 Remove Element LC #32 Move Zeroes
OOP with Java	10	Regular	OOP Fundamentals: Class, Object, Constructor, this keyword, static vs instance, access modifiers	Design a BankAccount class with accountNumber, holder, balance. Implement deposit(), withdraw() (with overdraft check), getStatement(). Add a static field to track total accounts created. Create 3 accounts and simulate 5 transactions each.	LC #33 Design Parking System LC #34 Design HashSet LC #35 Design HashMap
OOP with Java	11	Regular	Encapsulation & Abstraction: getters/setters, data hiding, abstract classes, abstract methods	Build a Shape hierarchy: abstract class Shape with abstract area() and perimeter(). Concrete classes: Circle, Rectangle, Triangle. Store shapes in an array and print a formatted area report. Enforce immutability on radius using private + getter only.	LC #36 Design Parking System LC #37 Min Stack LC #38 Implement Stack using Queues
OOP with Java	12	Regular	Inheritance: extends, super keyword, constructor chaining, method overriding, final keyword	A university system has Person → Student → GradStudent. Each level adds attributes (GPA, thesis). Override toString() at each level. Use super() to chain constructors. Make studentId final. Demonstrate that GradStudent IS-A Student IS-A Person.	LC #39 Binary Search Tree Iterator LC #40 Flatten Nested List Iterator



BRIDGELABZ JAVA PROGRAMMING FUNDAMENTALS IMMERSION MAKER PLAN

— 160 HRS

Stages - Core Java Programming, Object Oriented Programming with Java (OOP), Data Structure & Algorithms (DSA)

Core Programming (9 Days) | OOP with Java (11 Days) | DSA (15 Days) | 32 Sessions + 3 Workshops | Platform: CodIn

Fellowship Part	Day #	Session Type	Concept Introduced	Story-Based Sample Problem Statements (Daily Practice)	LeetCode Sample Problem References
OOP with Java	13	Regular	Polymorphism: runtime polymorphism, instanceof, object casting, dynamic method dispatch	A transport company manages a fleet: Vehicle[] containing Car, Bus, Bike objects. Call fuelCost(km) on each — each type calculates differently. Use instanceof before casting. Add a new ElectricCar without changing existing code (open/closed principle preview).	LC #41 Design Parking System LC #42 Encode and Decode Strings
OOP with Java	14-15	Regular	Interfaces: interface vs abstract class, multiple interface implementation, default & static interface methods	Build a fitness tracker: interfaces Trackable (logActivity()), Reportable (generateReport()), Notifiable (sendAlert()). Class FitnessDevice implements all three. Add a default method resetData() to Trackable. Demonstrate multiple interface implementation Java cannot do with classes.	LC #43 Implement Queue using Stacks LC #44 Implement Stack using Queues LC #45 LRU Cache (interface design)
OOP with Java	16-17	Regular	Collections Framework: ArrayList, LinkedList, HashMap, HashSet — CRUD operations and iteration	An address book app stores contacts (name, phone, email). Use ArrayList for ordered storage, HashMap<String,Contact> for fast name lookup, HashSet to prevent duplicate phone numbers. Implement: add, search, delete, display sorted by name using Collections.sort().	LC #46 Two Sum LC #47 Contains Duplicate LC #48 Intersection of Two Arrays LC #49 First Unique Character
OOP with Java	18-19	Regular	Generics: generic classes, generic methods, bounded type parameters, wildcards	Build a generic Pair<T, U> class and a generic Stack<T>. Write a generic method findMax(T[] arr) bounded to Comparable. Create a generic Repository<T> that stores any entity type. Demonstrate wildcard List<?> for a utility print method.	LC #50 Find Median from Data Stream (generic container) LC #51 Flatten Nested List Iterator



BRIDGELABZ JAVA PROGRAMMING FUNDAMENTALS IMMERSION MAKER PLAN
— 160 HRS

Stages - Core Java Programming, Object Oriented Programming with Java (OOP), Data Structure & Algorithms (DSA)

Core Programming (9 Days) | OOP with Java (11 Days) | DSA (15 Days) | 32 Sessions + 3 Workshops | Platform: CodIn

Fellowship Part	Day #	Session Type	Concept Introduced	Story-Based Sample Problem Statements (Daily Practice)	LeetCode Sample Problem References
OOP with Java	20	Workshop	WORKSHOP 2: OOP Design Challenge — Library Management System	Design a full Library Management System using OOP. Entities: Book (abstract), PhysicalBook, EBook; Member, PremiumMember (extends Member); interface Borrowable, Reservable. Use ArrayList + HashMap for storage, Search/filter & sorting. Must apply all OOP pillars.	LC #52 LRU Cache LC #53 LFU Cache LC #54 Design Parking System LC #55 Implement Trie LC #56 Encode Decode Strings
DSA	21	Regular	Sorting Algorithms: Bubble, Selection, Insertion Sort — swap counting, stability, best/worst case	A sports meet has 50 athletes with scores. Sort using Bubble Sort, count total swaps, and flag if already sorted (best case). Repeat with Insertion Sort. Find top-3 medalists. Trace pass-by-pass changes on array [64, 25, 12, 22, 11].	LC #57 Sort an Array LC #58 Sort Colors LC #59 Largest Number LC #60 Height Checker
DSA	22	Regular	Efficient Sorting: Merge Sort, Quick Sort, Counting Sort — divide-and-conquer, partitioning	A library sorts 1M books by year. Implement Merge Sort showing O (n log n) merge step. Quick Sort with Lomuto partition on 3-way pivot. Counting Sort for bounded categories (genre codes 1–20). Compare empirical runtime on sizes 100 / 1000 / 10000.	LC #61 Sort an Array LC #62 Count of Smaller Numbers After Self LC #63 Sort List LC #64 Reverse Pairs
DSA	23	Regular	Binary Search: standard, rotated arrays, first/last occurrence, search on answer	An astronomer has 10M sorted star-brightness entries. Binary search for a target in O(log n). Extend: search in rotated catalog (telescope glitch scrambled it), find first and last occurrence of a repeated brightness value, and find the minimum element.	LC #65 Binary Search LC #66 Search in Rotated Sorted Array LC #67 First and Last Position LC #68 Minimum in Rotated Array LC #69 Find Peak Element



BRIDGELABZ JAVA PROGRAMMING FUNDAMENTALS IMMERSION MAKER PLAN
— 160 HRS

Stages - Core Java Programming, Object Oriented Programming with Java (OOP), Data Structure & Algorithms (DSA)

Core Programming (9 Days) | OOP with Java (11 Days) | DSA (15 Days) | 32 Sessions + 3 Workshops | Platform: CodIn

Fellowship Part	Day #	Session Type	Concept Introduced	Story-Based Sample Problem Statements (Daily Practice)	LeetCode Sample Problem References
DSA	24	Regular	Two Pointers & Sliding Window: pair sum, triplets, max subarray of length k, variable window	A river station records hourly water levels. Find the longest contiguous period where levels stayed within a range (variable sliding window). Find all unique triplets of stations summing to zero pollution index. Max average subarray of length k.	LC #70 Container With Most Water LC #71 3Sum LC #72 Longest Substring Without Repeating LC #73 Minimum Window Substring LC #74 Permutation in String LC #75 Fruit Into Baskets
DSA	25-26	Regular	Linked List: singly & doubly — insert, delete, reverse, cycle detection, merge, find middle	A train management system models coaches as a linked list. Add/remove coaches, reverse the entire train, detect a circular route (cycle), find the middle coach for inspection, and merge two sorted train schedules into one sorted list.	LC #76 Reverse Linked List LC #77 Linked List Cycle LC #78 Middle of Linked List LC #79 Merge Two Sorted Lists LC #80 Remove Nth from End LC #81 Palindrome Linked List
DSA	27	Regular	Stacks: array & linked-list implementation, balanced brackets, next greater element, stock span	A compiler validates code for balanced brackets (), {}, []. Build stack-based validator. Extend: given daily stock prices, compute Next Greater Price for each day (O(n) monotonic stack). Calculate stock span (how many consecutive previous days price was $\leq$ today).	LC #82 Valid Parentheses LC #83 Daily Temperatures LC #84 Next Greater Element I LC #85 Largest Rectangle in Histogram LC #86 Decode String
DSA	28	Regular	Queues & Deque: circular queue, deque, priority queue (min-heap), sliding window maximum	A hospital triage system processes patients by severity (priority queue). Enqueue with urgency level, dequeue highest-urgency first. Extend: sliding window maximum on patient vitals using monotonic deque. Implement circular queue for a round-robin CPU scheduler.	LC #87 Sliding Window Maximum LC #88 Last Stone Weight LC #89 Top K Frequent Elements LC #90 Kth Largest Element in Array LC #91 Design Circular Deque



BRIDGELABZ JAVA PROGRAMMING FUNDAMENTALS IMMERSION MAKER PLAN
— 160 HRS

Stages - Core Java Programming, Object Oriented Programming with Java (OOP), Data Structure & Algorithms (DSA)

Core Programming (9 Days) | OOP with Java (11 Days) | DSA (15 Days) | 32 Sessions + 3 Workshops | Platform: CodIn

Fellowship Part	Day #	Session Type	Concept Introduced	Story-Based Sample Problem Statements (Daily Practice)	LeetCode Sample Problem References
DSA	29	Regular	Hashing: HashMap internals, HashSet, collision handling, frequency map, frequency problems, two-sum variants, subarray sum	A detective agency tracks suspects across cases. HashMap: suspect → case list. Find the most investigated suspect, all suspects in exactly 2 cases, and every pair of suspects always seen together. Extend: find subarray with sum equal to target K using prefix sum + HashMap.	LC #92 Two Sum LC #93 Group Anagrams LC #94 Longest Consecutive Sequence LC #95 Subarray Sum Equals K LC #96 LRU Cache
DSA	30	Regular	Trees: Binary Tree traversals (recursive + iterative), (in/pre/post/level), height, diameter, level-order BFS	A family tree app stores ancestors. Implement all 4 traversals recursively and iteratively (using stack/queue). Find height, count nodes at each level (level-order BFS), compute diameter (longest path between any two relatives), and check if balanced.	LC #97 Tree Traversals LC #98 Level Order Traversal LC #99 Diameter of Binary Tree LC #100 Balanced Binary Tree LC #101 Invert Binary Tree LC #102 Maximum Depth
DSA	31	Regular	BST: insert, search, delete (all 3 cases), validation, kth smallest, LCA	A dictionary app stores words in a BST for O(log n) lookup. Insert 20 words, search, delete covering all 3 node cases, find the kth alphabetically (in-order), validate the BST property, and find Lowest Common Ancestor of two words.	LC #103 Search in BST LC #104 Validate BST LC #105 Kth Smallest in BST LC #106 LCA of BST LC #107 Delete Node in BST LC #108 Sorted Array to BST
DSA	32	Regular	Heaps: min/max heap, heapify, heap sort, top-K problems	A stock exchange ranks companies by market cap. Build a max-heap from scratch (heapify). Use Java's PriorityQueue to find top-K performing stocks. Implement heap sort.	LC #109 Kth Largest Element LC #110 Merge K Sorted Lists LC #111 Kth Smallest in Sorted Matrix LC #112 Find Median from Data Stream

		BRIDGELABZ JAVA PROGRAMMING FUNDAMENTALS IMMERSION MAKER PLAN — 160 HRS			Stages - Core Java Programming, Object Oriented Programming with Java (OOP), Data Structure & Algorithms (DSA)
Core Programming (9 Days) OOP with Java (11 Days) DSA (15 Days) 32 Sessions + 3 Workshops Platform: CodIn					
Fellowship Part	Day #	Session Type	Concept Introduced	Story-Based Sample Problem Statements (Daily Practice)	LeetCode Sample Problem References
DSA	33	Regular	Graphs: adjacency list & matrix, BFS shortest path, DFS all paths, connectivity, cycle detection	A city navigation app models roads as a graph of 10 cities. BFS: find shortest path (fewest stops) between any two cities. DFS: enumerate all paths from source to destination. Detect connected components. Detect cycle in directed graph (course prerequisite check).	LC #113 Number of Islands LC #114 Clone Graph LC #115 Course Schedule LC #116 Rotting Oranges LC #117 Pacific Atlantic Water Flow LC #118 Surrounded Regions
DSA	34	Regular	Recursion & Backtracking: N-Queens, combination sum, permutations, subsets, word search	A chess puzzle generator places N queens on N×N board without conflicts using backtracking with pruning. Generate all permutations of a secret code, all subsets of inventory items, and find all valid combinations summing to a target budget.	LC #119 N-Queens LC #120 Combination Sum LC #121 Permutations LC #122 Subsets LC #123 Word Search LC #124 Generate Parentheses LC #125 Combination Sum II
DSA	35	Workshop	WORKSHOP 3: DSA	3-hour workshop on CodIn. 8 problems — Easy (2), Medium (4), Hard (2) — spanning Arrays, Linked List, Trees, Graphs, DP, Backtracking. Scoring: correctness + speed. Peer code review + mentor feedback session.	LC #126 Trapping Rain Water LC #127 Largest Rectangle in Histogram LC #128 Merge K Sorted Lists LC #129 Binary Tree Max Path Sum LC #130 Course Schedule LC #131 Burst Balloons LC #132 N-Queens LC #133 Find Median from Data Stream

PROGRAM SUMMARY	
Total Duration	160 Hours 35 Days 32 Regular Sessions + 3 Workshops

		BRIDGELABZ JAVA PROGRAMMING FUNDAMENTALS IMMERSION MAKER PLAN — 160 HRS			Stages - Core Java Programming, Object Oriented Programming with Java (OOP), Data Structure & Algorithms (DSA)
Core Programming (9 Days) OOP with Java (11 Days) DSA (15 Days) 32 Sessions + 3 Workshops Platform: CodIn					
Fellowship Part	Day #	Session Type	Concept Introduced	Story-Based Sample Problem Statements (Daily Practice)	LeetCode Sample Problem References
Part 1: Core Programming		Days 1–9 8 Sessions + 1 Workshop ~45 hrs Java Basics, Control Flow, Arrays, Strings, Recursion, Exception Handling, File I/O			
Part 2: OOP with Java		Days 10–20 10 Sessions + 1 Workshop ~55 hrs OOP Pillars, Interfaces, Collections, Generics			
Part 3: DSA		Days 21–35 14 Sessions + 1 Workshop ~60 hrs Sorting, Binary Search, Linked List, Stacks, Queues, Trees, Graphs, Backtracking			
Total Problems		~230 Daily Practice Problems + 100+ LeetCode Problem References			